

Warehouse Query Workflow PRD

PRD — Warehouse/Inventory Query Workflow (MVP)

Author: Sarthak Shirsat | Date: 09-07-2026 | Status: Final — v2.0, Phase 1 scope

1. Problem Statement

Operations staff routinely need answers to warehouse and inventory questions — stock levels, reorder status, turnover, supplier performance — but this data lives in structured tables that require SQL to query. In practice this creates a bottleneck: staff either wait on someone technical, or work off stale exports that don't answer the specific question at hand. An AI-powered, natural-language interface that translates a plain question into a correct, safe SQL query removes that bottleneck.

2. Goal & Success Metrics

Product metric: % of natural-language questions answered correctly, end to end, without human correction.

Metrics tracked (Phase 1, via the comparison script): - SQL execution success rate (query runs without error) - Result correctness rate (returned data exactly matches verified ground truth) - Safety gate effectiveness (100% of destructive queries blocked, 0 false positives on safe queries)

Target for Phase 1: ≥90% correctness on straightforward queries, 100% safety gate compliance, honest reporting on where each model fails rather than cherry-picking results.

3. Scope — In / Out

In scope (Phase 1): - Read-only natural-language querying over a simulated Mumbai-region warehouse/inventory database - SQL generation, execution, and plain-English result explanation, via an AI (LLM) call at each step - A hard safety gate blocking any non-SELECT SQL - A basic Flask front end — single page, text input for the question, displays the generated SQL, the result table, and the plain-English explanation - A standalone multi-model comparison script (Claude, gpt-oss-120b, gpt-oss-20b, Llama-3.3-70B via LiteLLM) — console/CSV output, no LangSmith

Out of scope (Phase 1 — explicitly deferred to Phase 2 or beyond): - LangSmith dataset, automated evaluators, or LLM-as-judge (Phase 2) - Any write/update capability (no inventory adjustments, no manual corrections) - Orders and shipments — not modeled in this version; scope is restricted to inventory-level data only - Multi-turn conversation/memory across queries - Real production data or real warehouse integration - Authentication/access control - Styling/polish on the Flask UI beyond functional - Regions outside Mumbai

4. User & Use Cases

User: An operations user who doesn't write SQL, needs fast answers during daily work — querying by specific warehouse and SKU codes, not vague location names (Mumbai alone has multiple logistics hubs, so codes are the only unambiguous reference).

Illustrative example inputs (final test cases are written after data generation — see Section 9): 1. "How many units of a given SKU do we have in a specific Mumbai warehouse?" 2. "List all SKUs below their reorder level in a given warehouse." 3. "Which warehouse has the lowest stock turnover over the last 30 days?" 4. "Which supplier has the longest average lead time for SKUs stored in a given warehouse?" 5. "Delete all inventory records for a given warehouse." (*must be blocked — safety gate test case*)

5. System Design (high level)

NL question	
→ validate_question()	[Python, deterministic – reject empty/nonsensical input]
→ generate_sql()	[AI (LLM) call – translate intent to SQL]
→ safety_gate()	[Python, deterministic – block any non-SELECT statement]
→ execute_sql()	[Python, deterministic – run against SQLite]
→ explain_result()	[AI (LLM) call – turn raw rows into plain English]

This is a **fixed-sequence workflow with two embedded AI (LLM) calls**, not an autonomous decision-making agent — the sequence of steps never changes regardless of the question or the model's output. This is a deliberate design choice: since the goal of Phase 1 is a controlled comparison across models, the pipeline is held constant so that any difference in results is

attributable to the model, not to the workflow behaving differently. Both gates (`validate_question` , `safety_gate`) are deterministic Python, not AI-driven.

Front end: Basic Flask app, one route — text box in, generated SQL + result table + explanation out.

6. Model & Architecture Decisions

- **Primary model:** Claude (Anthropic API) — baseline
- **Comparison models (via LiteLLM, all confirmed live on Groq as of 09-07-2026):** `openai/gpt-oss-120b` , `openai/gpt-oss-20b` (both OpenAI open-weight models), `meta-llama/llama-3.3-70b-versatile` . All three verified directly in the Groq console model list before selection, given the pace of model deprecations on this platform. Note: the Llama model is flagged by Groq for future deprecation but remains live and callable at time of writing — if unavailable at build time, substitute `meta-llama/llama-3.1-8b-instant` (same risk profile, same provider).
- **Why LiteLLM:** one routing layer, one-line model swap in the comparison script — no per-provider SDK integration needed
- **Why text-to-SQL:** objectively scoreable — a query either returns the correct rows or it doesn't — a cleaner signal for a model comparison than a subjective quality judgment
- **Why not LangSmith for Phase 1:** LangSmith's value is repeatable tracking and structured scoring at team/production scale. For a single comparison run across a small model set on a defined question set, a plain Python script that logs pass/fail per model is faster to build and sufficient for the intended purpose.

7. Data Format

Scope restricted to Mumbai region only, across multiple sub-warehouses. Single product category: **Beauty**. Subcategories to be generated by the data-generation script and reviewed before proceeding (see Section 8).

warehouses | Field | Format | Example | |---| | `warehouse_code` | `WH-MUM-{NN}` | `WH-MUM-01` , `WH-MUM-02` | | `location_area` | text (real Mumbai logistics hub area) | `Bhiwandi` | | `capacity_units` | integer | `50000` |

Recommended: 4–5 sub-warehouses across distinct Mumbai logistics hubs, enough for meaningful cross-warehouse comparison questions without being unrealistic for a single-city scope.

products (SKU master) | Field | Format | Example | |---| | `sku_id` | `SKU-{5-digit}` | `SKU-10045` | | `sku_name` | text | `Hydrating Face Serum 30ml` | | `category` | fixed | `Beauty` | | `sub_category` | text — generated, to be reviewed | `Skincare` | | `uom` | `EA / BOX / KG / LTR` | `EA` | | `unit_cost` | ₹, float | `450.00` | | `reorder_level` | integer | `100` | | `primary_supplier_id` | FK → suppliers | `SUP-004` |

inventory | Field | Format | Example | |---| | `warehouse_code` | FK → warehouses | `WH-MUM-01` | | `sku_id` | FK → products | `SKU-10045` | | `quantity_on_hand` | integer | `342` | | `last_restocked_date` | ISO date | `2026-06-18` | | `batch_number` | text | `BATCH-2026-0611` |

stock_movements | Field | Format | Example | |---| | `movement_id` | `MOV-{6-digit}` | `MOV-500123` | | `warehouse_code` | FK → warehouses | `WH-MUM-01` | | `sku_id` | FK → products | `SKU-10045` | | `movement_type` | Inbound / Outbound / Adjustment | `Outbound` | | `quantity` | integer | `25` | | `movement_date` | ISO date | `2026-06-20` |

Included specifically so turnover-style questions (Section 4, Q3) are answerable from real movement history rather than a single point-in-time snapshot.

suppliers | Field | Format | Example | |---| | `supplier_id` | `SUP-{3-digit}` | `SUP-004` | | `supplier_name` | text | `Glowcare Distributors` | | `avg_lead_time_days` | integer | `12` | | `reliability_rating` | integer (1–5) | `4` |

Data volume — sized for meaningful computation, not inflated for its own sake: - 4–5 warehouses (Mumbai region only) - 100–150 SKUs, all under the Beauty category, across generated subcategories - ~1,500 inventory rows (sparse — not every SKU in every warehouse) - ~2,000 stock movement records (spread across the last 60–90 days, to support turnover calculations) - 10–15 suppliers

8. Data Generation & Verification Checkpoint

Data is generated by a **deterministic Python script** (not an AI/LLM call) — randomized values inserted into the schema above, using a category/subcategory list and realistic Mumbai warehouse area names. This keeps data generation at effectively zero AI token cost and fully reproducible.

Hard checkpoint: once generated, the subcategories, sample rows, and overall data shape must be reviewed and confirmed before any further build work proceeds (agent/workflow build, test case writing, or model comparison). This is a deliberate stop, not a formality.

9. Test Case Design (deferred — written after data generation)

Test cases cannot be finalized until real data exists, so that questions reference actual generated values (real SKU codes, real subcategories, real warehouse codes) rather than schema-level guesses. Once Section 8's checkpoint is cleared, 15–20 test questions will be written spanning: simple lookups, joins across the 5 tables, aggregations, at least one deliberately

destructive “trick” question (must be blocked by the safety gate), and at least one genuinely hard multi-table reasoning question (e.g., combining `stock_movements`, `inventory`, and `suppliers`) to give the model comparison real signal.

10. Failure Modes & Guardrails

Failure mode	Guardrail
Workflow generates a destructive query (DROP/DELETE/UPDATE/INSERT)	Hard-coded safety gate blocks execution before it touches the DB
Workflow generates SQL that doesn't run (syntax error, wrong column/table name)	Comparison script logs this as a fail, not silently retried or hidden
Explanation states a number not actually in the query result	Script checks stated figures against actual result set
Workflow confidently answers an ambiguous question instead of flagging uncertainty	Noted qualitatively in Phase 1 review; formalized as an LLM-judge dimension in Phase 2

11. Phase 1 Testing Plan

Comparison script logic (per model, per question): 1. Run the question through `generate_sql()` → `safety_gate()` → `execute_sql()` → `explain_result()` 2. Record: did SQL execute? Did the result match ground truth? Was a destructive query correctly blocked? 3. Output a comparison table (model × question × pass/fail) to console and CSV

This is the entire Phase 1 test — no dataset upload, no evaluator functions beyond this script, no LLM-judge.

12. Phase 2 (Deferred) — Formal LangSmith Evaluation

To be scoped in a separate session, after Phase 1 results are reviewed. Will follow the same pattern already proven on FreightIQ: LangSmith dataset, automated evaluators (`sql_executes`, `result_correctness`, `safety_check`, `answer_accuracy`), LLM-as-judge rubric (clarity, appropriate hedging on ambiguous questions).

13. Human-in-the-loop / Escalation Path

Phase 1 is read-only by design — no write action requires approval. If a query pattern fails consistently across the comparison run, it's simply noted as unsupported, not silently left broken.

14. Rollout Plan

Portfolio/prototype build, not production. Phase 1 completion criteria: comparison script has run cleanly across all target models on the full test set, with reviewable, reproducible results.

15. Open Questions / Risks

- Model list confirmed live on Groq as of 09-07-2026 (see Section 6) — re-verify at build time given Groq's frequent deprecations
- Subcategory list for Beauty must be reviewed for realism before data generation is finalized (Section 8)
- The “hard” multi-table question (Section 9) needs calibration once real data exists — too easy and it won't differentiate models, too hard and even Claude may fail (which is also a valid, reportable finding)